

Week 4 Module – Programming Practice (3rd Year)

Date: 09/07/2026

Language Used: JAVA

Pdf follows: handwritten code with logic and followed by the respective outputs.

Submission details:---

Name : Sakshi das

Roll number : 2404921520194

Task completed on : 14/07/2026 – Wednesday

Submitted on : 16/07/2026

complexity analysis

Week 4 module

Module 1: Patterns.

A) Star (1) Square star Pattern.

Problems Time complexity : $O(N^2)$.

space complexity : $O(1)$.

The program uses two nested loops.

The outer loop $\Rightarrow$ N rows.

inner loop $\Rightarrow$ N stars in each row.

$\therefore N \times N = N^2 \Rightarrow O(N^2)$.

(2) Hollow square Pattern.

Time complexity : $O(N^2)$.

space complexity : $O(1)$.

The program checks ~~to~~ every position to decide whether to print a star or a space.

(3) Right Triangle Pattern.

Time complexity : $O(N^2)$

space complexity : $O(1)$.

outer loop : N times

inner loop : $\frac{N(N+1)}{2} \approx N \text{ times} \Rightarrow O(N^2)$.

(4) Inverted Triangle Pattern.

Time complexity : $O(N^2)$

space complexity : $O(1)$.

The no. of stars decreases in each row.

(5) Pyramid Pattern :

Time complexity : $O(N^2)$

space complexity : $O(1)$.

Nested loops used here. Each row prints spaces followed by stars.

(6) Diamond Pattern.

Time complexity: $O(N^2)$

Space complexity: $O(1)$.

Both halves use nested loops. The characters ~~use~~ ^{get} ~~pro~~ is proportional to $O(N^2)$.

(7) Half Diamond Pattern.

Time complexity: $O(N^2)$

Space complexity: $O(1)$.

(8) Inverted Half Diamond Pattern.

Time complexity: $O(N^2)$

Space complexity: $O(1)$.

(B) Number

Pattern: 1) Square Number Pattern.

Time complexity: $O(N^2)$

Space complexity: $O(1)$.

~~similar logic and analysis done as done on star problems.~~

Two nested loops are used to print the no.

The outer loop = n rows.

The inner loop = n numbers. $\Rightarrow O(N^2)$.

(2) Incrementing number pattern.

Time complexity: $O(N^2)$

Space complexity: $O(1)$.

(similar logic and analysis were done as in square no. pattern).

(3) Right Triangle number pattern.

Time complexity: $O(N^2)$

Space complexity: $O(1)$.

(similar logic and analysis were done as in square number pattern).

14) Inverted ~~triangle~~^{Diamond} number pattern.

Time complexity: $O(N^2)$

Space complexity: $O(1)$.

(similar logic & analysis were done as in square no. pattern).

~~15~~ Diamond number pattern. (incremented Half Diamond).

Time complexity: $O(N^2)$.

Space complexity: $O(1)$.

(Simple similar logic & analysis were done as in square no. pattern).

~~16~~ sandwich pattern:

Time complexity: $O(N^2)$

Space complexity: $O(1)$.

(similar logic & analysis were done as in square no. pattern).

~~17~~

Module 2: Searching & Sorting.

1) Linear Search: Fraud Transaction Detection.

The searching technique checks each Transaction ID one by one until the suspicious ID is found out.

The algorithm only uses a few variables, so the extra memory required is constant.

Time complexity: ~~$O(N)$~~ $O(1)$ Best case,

$O(N)$ Average & worst case

Space complexity: $O(1)$.

2) Binary ~~log~~ search : Binary log search.

Time complexity: $O(1)$ Best case, $O(\log N)$ Average & Worst case.
Space complexity: $O(1)$.

Binary search works on sorted data. It repeatedly compares the target log ID with the middle element & eliminates half of the remaining search.

That significantly reduces the no. of comparisons giving a logarithmic running time.

(3) Bubble sort.

Time complexity: $O(N^2)$ Best case, $O(N^2)$ Average & Worst case.

Space complexity: $O(1)$.

The sorting technique compares adjacent salary values and swaps them if they are in wrong order.

(4) Insertion sort.

Time complexity: $O(N)$ Best case, $O(N^2)$ Average & Worst case.

Space complexity: $O(1)$

When the data is nearly sorted, very few shifts are required, making the algorithm efficient.

(5) Selection sort.

Time complexity: $O(N^2)$. Best, Average, & Worst case.

Space complexity: $O(1)$.

Only a few variables were used for swapping so the space complexity is constant.

Since it scans the remaining elements during every pass regardless of i/p order

$\therefore$ its running time remains $O(N^2)$.

Module 3: Advanced sorting.

(1) merge sort (Easy).

Time complexity: $O(N \log N)$. (Best, Average & worst case)

Space complexity: $O(N)$.

Since the array is divided into $\log N$ & each level processes N elements, then +

$\therefore$ time complexity = $O(N \log N)$.

(2) merge sort (medium).

Time complexity: $O(N \log N)$

Space complexity: $O(N)$.

(Similar approach & analysis were done as in Q1).

(3) Quick sort (Easy).

Time complexity: $O(N \log N)$. (Best & Average case)

$O(N^2)$. (worst case).

Space complexity: $O(\log N)$.

(4) Quick sort (medium)

Time complexity: $O(N \log N)$.

Space complexity: $O(\log N)$.

(5) Heap sort (Easy).

Time complexity: $O(N \log N)$ (Best, Average & worst case)

Space complexity: $O(1)$.

Heap sort operations take $\log N$ time, and they are performed for all N elements.

(6) Heap sort (medium).

Time complexity: $O(N \log N)$

Space complexity: $O(1)$.

Section - B

Q1) Print numbers from 1 to N.

```
import java.util.Scanner;  
public class PrintNumbers {  
    public static void main(String[] args) {  
        Scanner sc = new Scanner(System.in);  
        int n = sc.nextInt();  
        for (int i = 1; i <= n; i++) {  
            System.out.print(i + " ");  
        }  
        sc.close();  
    }  
}
```

Time complexity: $O(N)$

Space complexity: $O(1)$

The program uses a single loop (1 to N) $\rightarrow$ printing each number exactly once.

$\therefore$ no. of operations increases linearly with i/p size.

* Only a few variables were used, so the extra memory required is constant.

12) Sum of First N natural no.

```
import java.util.Scanner;  
public class sum {  
    public static void main (String[] args) {  
        Scanner sc = new Scanner (System.in);  
        int n = sc.nextInt();  
        long sum = 0;  
        for (int i = 1; i <= n; i++) {  
            sum += i;  
        }  
        System.out.println(sum);  
        sc.close();  
    }  
}
```

Time complexity: $O(N)$
Space complexity: $O(1)$.

The loop runs exactly N times and adds each natural no. to the variable (sum).
Since, each no. is processed once $\Rightarrow$ ~~sum~~ N.

$$\text{sum} = \frac{N(N+1)}{2} \approx N.$$

(3) i/p : 25

$\Rightarrow 2^5 \Rightarrow 32$ o/p.

```
import java.util.Scanner;
public class Power {
    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);
        int a = sc.nextInt();
        int b = sc.nextInt();
        long result = 1;
        for (int i = 1; i <= b; i++) {
            result = result * a;
        }
        System.out.println(result);
        sc.close();
    }
}
```

3.

Time complexity: $O(B)$

Space complexity: $O(1)$

loop executes for B times, multiplying the result by A.

$\therefore$ the running time depends on the ~~val~~ value B. (superscript).

*Extra memory remains constant.